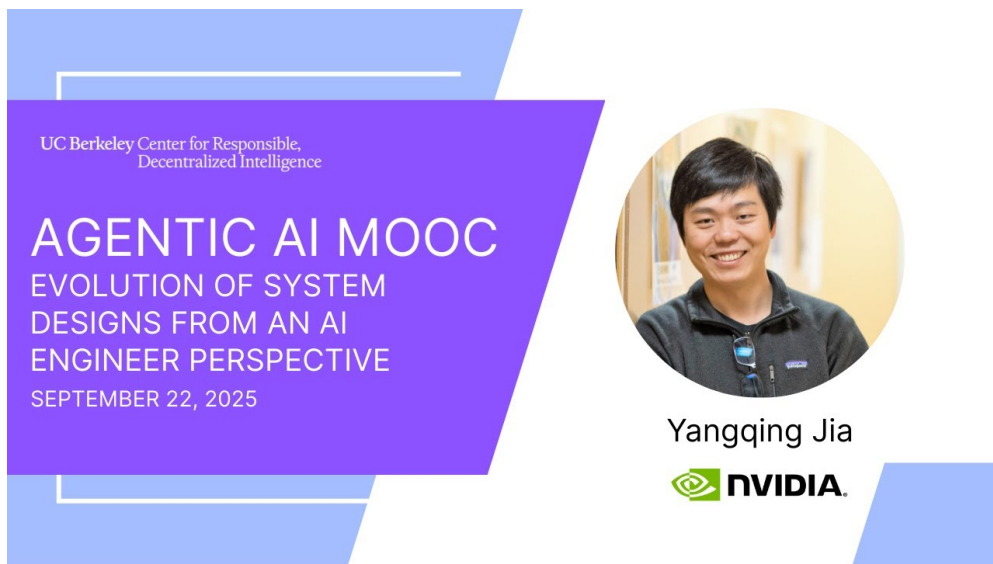


课程笔记

[LLM Agents F25] Evolution of System Designs — Yangqing Jia

基于 Yangqing Jia 授课内容整理

2026-04-02



视频作者/频道: Berkeley RDI

发布日期: 2025-09-26

视频时长: 1:19:31

视频链接: <https://www.youtube.com/watch?v=xqRAS6rAouo>

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1 课程定位：从模型能力竞争转向系统能力竞争	3
1.1 主讲人视角与问题定义	3
1.2 本章小结	3
2 第一性原理：从“神秘 AGI”到“可分解系统”	3
2.1 输入法类比：复杂智能可由简单模块组合	3
2.2 本章小结	4
3 技术浪潮切换：MLOps、RAG 与 Agentic AI	4
3.1 热度演化与真实需求	4
3.2 消费应用与企业应用的分化	5
3.3 阶段能力对比	6
3.4 本章小结	6
4 基础设施谱系：HPC、Cloud、Data Cloud 到 AI Cloud	6
4.1 从科学计算到通用云	6
4.2 Web 2.0 到 Data Cloud	6
4.3 AI Cloud 的新矛盾：IO 与数值计算	7
4.4 本章小结	8
5 生产系统中的脆弱点：故障风暴与恢复设计	8
5.1 大型 Chatbot 事故案例	8
5.2 15 分钟恢复的关键动作	9
5.3 本章小结	9
6 硬件与系统共设计：从 OCP 到 DGX	9
6.1 架构回摆：从分散微服务到高密度计算节点	9
6.2 节点形态对软件栈的影响	10
6.3 本章小结	10
7 RAG 与 Agent 的关系：不是替代，而是内化	10
7.1 RAG “降温”的真实含义	10
7.2 分阶段排序：成本与准确率的联合优化	11
7.3 幻觉问题的边界：常识与事实的分界线	11
7.4 本章小结	12
8 AI 云调度难题：拓扑、数据与抽象层冲突	12
8.1 局部性约束重新变成一等公民	12
8.2 数据面复杂度：不只是“装个框架”	12
8.3 本章小结	13

9 创业与产业判断：价值优先、垂直突破、效率回归	13
9.1 创业切入点：基础模型之外的价值密度	13
9.2 中心化与边缘化的动态平衡	13
9.3 价值创造与风险边界	14
9.4 本章小结	14
10 工程落地清单：面向 Agent 系统的设计原则	15
10.1 从课程内容抽象出的十条实践原则	15
10.2 评估框架	15
10.3 本章小结	16
11 案例推演：企业级 Agent 平台的分阶段建设路线	16
11.1 阶段 0：先做“可控的最小系统”	16
11.2 阶段 1：把检索与执行解耦	16
11.3 阶段 2：引入流控与恢复策略	16
11.4 阶段 3：治理与合规上线	17
11.5 本章小结	17
12 术语与指标附录：把系统讨论落到可执行口径	17
12.1 关键术语对照	17
12.2 建议的周报指标模板	18
12.3 本章小结	18
13 总结与延伸	19
13.1 全讲核心结论	19
13.2 进一步阅读	19

1 课程定位：从模型能力竞争转向系统能力竞争

1.1 主讲人视角与问题定义

这讲并不是在讲某个单点模型技巧，而是从系统工程历史去解释一个更大的命题：当大模型和 Agent 成为主流工作负载时，整个基础设施设计为什么必须重写。Yangqing Jia 的经历横跨 Google 研究、Caffe/TensorFlow/PyTorch 生态、Lepton AI 创业与被并入 NVIDIA 的工程实践，因此他的观察并非学术抽象，而是来自大规模生产系统中的成本、可靠性与演进压力。

本讲的核心问题

我们今天讨论的不是“Agent 是不是新概念”，而是“当 Agent 工作负载进入真实生产环境，哪些系统设计假设会失效，哪些旧原则会回归，哪些新抽象必须补上”。

课程从三个层次递进：

1. 技术浪潮层：MLOps → RAG → Agentic AI 的热度切换与真实留存；
2. 基础设施层：HPC、Cloud、Data Cloud、AI Cloud 的架构约束如何变化；
3. 产业落地层：企业应用、创业机会、边缘部署、成本与价值之间如何权衡。

为什么“系统设计”在 Agent 时代更关键

传统 LLM 应用可以停留在“单轮请求-响应”。Agent 系统则包含任务拆解、状态管理、外部工具调用、长链路执行、失败恢复与审计闭环。模型只是其中一个部件，真正决定能否商用的是系统能否稳定承载这些环节。

常见误区：把 Agent 仅理解为 Prompt 工程升级

如果只在提示词层面做“更复杂的模板”，没有补齐调度、存储、检索、流控、可观测性与故障恢复，那么系统会在高并发和长任务下快速失稳。很多所谓“模型不够强”的问题，实际是系统设计不足导致的。

1.2 本章小结

本讲的价值在于把 Agent 讨论从概念热度拉回工程现实：模型能力提升只是起点，系统演进才是规模化落地的决定因素。

2 第一性原理：从“神秘 AGI”到“可分解系统”

2.1 输入法类比：复杂智能可由简单模块组合

Yangqing Jia 在开场强调，外界常把 AGI 描述为神秘黑箱，但工程视角更强调可分解性。他用中文输入法做类比：用户输入拼音后，系统并不是“一次性神谕输出”，而是经历切分、候选召回、排序、语言模型修正等分层模块，再由交互反馈不断改进体验。

这个类比指向一个关键判断：大模型系统同样可以被拆为相对清晰的子问题。即使底层模型参数巨大，上层产品与基础设施仍然需要可解释的模块边界，否则无法调试、优化与治理。

工程含义：把“智能”还原为“流水线”

输入法并不需要“理解一切”才能工作良好，它依赖高质量候选生成与排序。Agent 系统也类似，不必追求单模型包办所有功能，而应优先把召回、规划、执行、验证各阶段做到可控和可观测。

输入法类比映射到 Agent Pipeline

- 拼音切分 ↔ 任务解析 (Task Parsing);
- 候选字召回 ↔ 工具/知识候选检索 (Retrieval);
- 候选排序 ↔ 规划与动作选择 (Planning / Policy);
- 用户纠错 ↔ 在线反馈与后训练闭环 (Feedback Loop)。

不要把“可分解”误解为“简单”

可分解并不意味着成本低。相反，模块越多，跨模块协议、数据一致性与故障联动风险越高。工程团队必须在“模块化收益”与“系统复杂度”之间做严谨权衡。

2.2 本章小结

“神秘能力”在工程上最终要落到“可分解系统”。这不是降低难度，而是把难点从模型单点，转移到跨模块协同与系统可靠性。

3 技术浪潮切换：MLOps、RAG 与 Agentic AI

3.1 热度演化与真实需求

在约 20 分钟处，讲者通过产业观察指出，MLOps、RAG、Prompt Engineering、Agentic AI 的热度迭代非常快。某些概念在一年内就从“新范式”走向“基础配置”。这并不意味着其失效，而意味着其被吸收进默认工程栈。

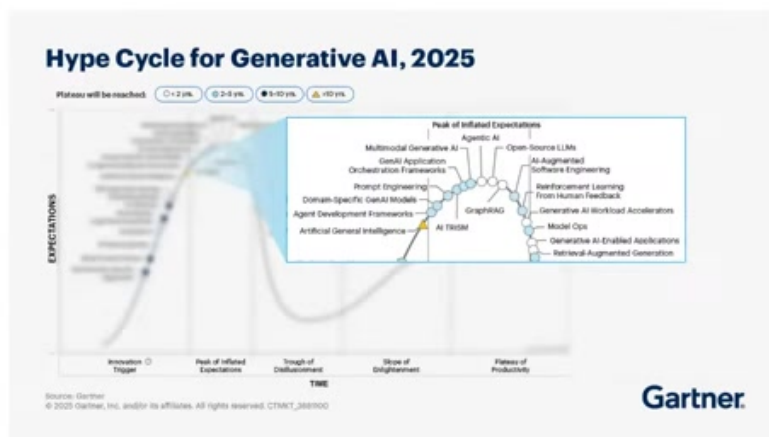


图 1: 关于技术热度迁移与新焦点 (Prompt Engineering / Agentic AI) ¹

热度不等于价值，沉淀才等于价值

RAG 的讨论热度下降，不代表 RAG 被淘汰；很多时候是因为它已成为系统默认能力，像数据库或缓存一样被隐式调用。

3.2 消费应用与企业应用的分化

讲者把应用分为两类：

- Consumer 场景：强调创意、娱乐、生产力，容忍一定不确定性；
- Enterprise 场景：强调正确性、稳定性、可追责，容忍错误的空间小得多。

这一区分决定了系统设计重点。消费场景常优先“体验增益”，企业场景必须先解决“可信交付”。同一模型在两类场景中，工程栈和治理要求差异极大。

同一模型，不同系统责任边界

在消费产品里，系统更多是“加速器”；在企业系统里，系统是“安全阀”。前者可偏向试错，后者必须先构建保守默认、审计日志和回滚机制。

错误迁移：把消费应用策略直接复制到企业场景

许多团队在 PoC 阶段成功后直接推广到企业生产，忽视权限、合规、事实校验和 SLA 约束，最终导致“演示可行、生产不可用”。

¹ 视频时间：00:20:07-00:20:13。

3.3 阶段能力对比

阶段	核心能力	主要瓶颈	系统重点
MLOps 阶段	模型训练/部署流水线	实验管理与发布效率	平台化与自动化
RAG 阶段	外部知识接入与问答增强	召回质量与延迟波动	检索链路 with 缓存策略
Agent 阶段	规划、工具调用、长任务执行	长链路稳定性与可观测性	编排、流控、审计与恢复

表 1: 从 MLOps 到 Agent 的系统责任扩张

3.4 本章小结

浪潮切换的本质不是概念替换，而是能力沉淀与系统责任扩张。Agent 时代把系统设计推到前台。

4 基础设施谱系：HPC、Cloud、Data Cloud 到 AI Cloud

4.1 从科学计算到通用云

讲者回顾了早期 HPC 集群时代：高算力任务（物理、天气等）运行在集中式集群上，强调并行计算能力。随后 VPS 与公有云出现，把硬件运维抽象出去，让开发者以更门槛部署业务。

公有云成功的抽象前提

机器被视作可替换资源，工作负载多为松耦合服务。只要接口一致，底层物理位置通常不影响业务逻辑。

4.2 Web 2.0 到 Data Cloud

随着用户行为数据激增，系统重心从“托管服务”走向“数据分析与推荐”。讲者以 Snowflake 与 Databricks 为代表，指出 Data Cloud 时代核心是大规模数据处理与 SQL/批流计算统一。

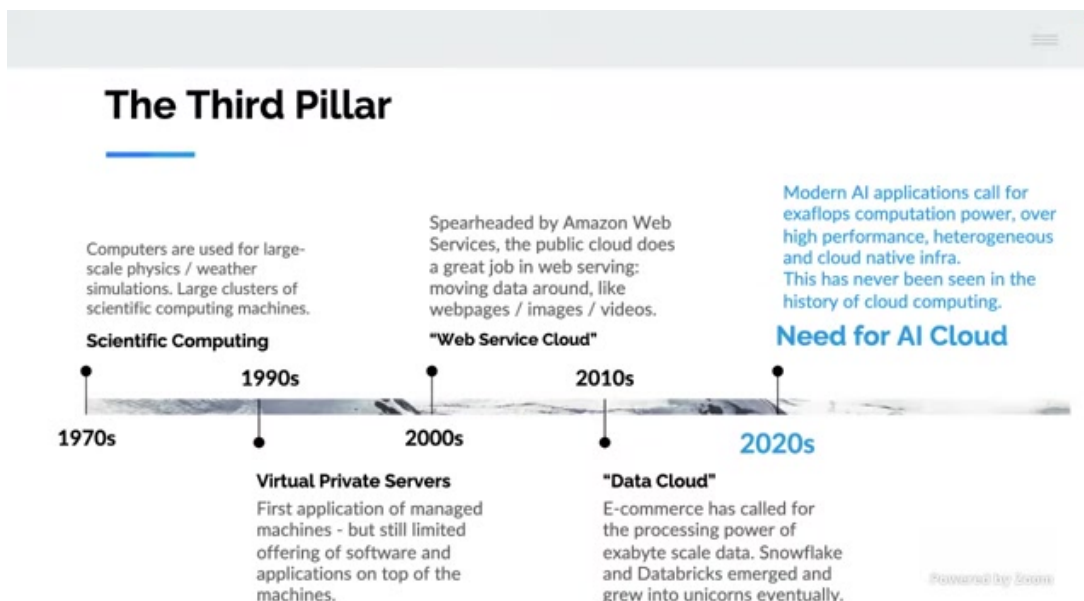


图 2: 从传统云演化到数据云与大规模分析栈²

4.3 AI Cloud 的新矛盾: IO 与数值计算

进入 LLM 时代后, 系统重新面临“数据移动”与“数值计算”的双重压力。讲者在 37 分钟附近明确提出, 基础设施演化可抽象为两条主轴: IO (数据搬运) 与 Compute (数值计算)。两者耦合方式决定架构上限。

Why? AI is different from conventional compute

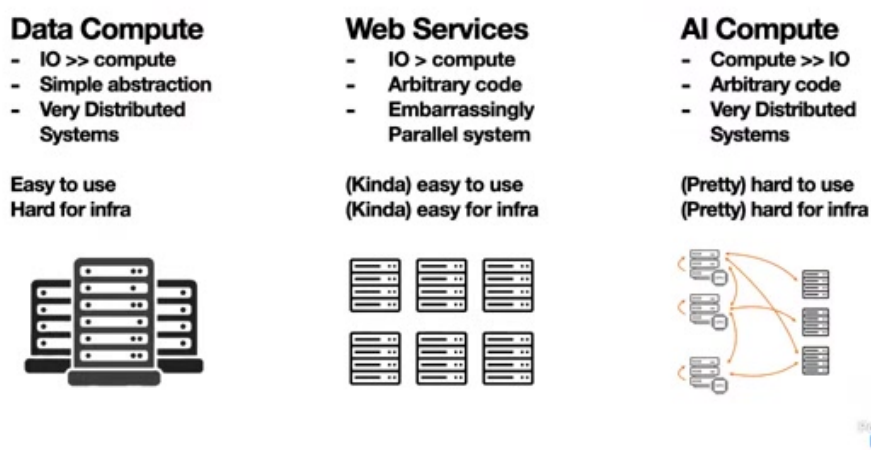


图 3: AI 基础设施的两大主轴: IO 与 Compute³

²视频时间: 00:36:33-00:36:46。

³视频时间: 00:37:29-00:37:36。

AI Cloud 的工程结论

不是“算力越大越好”，而是“算力、通信、存储、调度是否协同”。任何单点拉满都会在链路其他位置形成瓶颈。

盲目复用旧云抽象的风险

把 AI 训练/推理任务完全当作传统微服务调度，常导致跨机通信抖动、数据加载拥塞和任务碎片化，最终表现为算力利用率低与成本失控。

4.4 本章小结

AI Cloud 不是传统云的线性升级，而是回到“计算-通信-数据”一体化优化问题。系统设计必须从工作负载本身出发。

5 生产系统中的脆弱点：故障风暴与恢复设计

5.1 大型 Chatbot 事故案例

在约 48 分钟，讲者分享了一个真实事故：客户误关闭主模型服务后，系统重启触发请求堆积，首个恢复副本被突发流量打死，随后第二个副本重复失败，形成典型恢复风暴（restart storm）。



图 4: 大规模服务恢复阶段的流量风暴问题⁴

恢复阶段往往比故障发生阶段更危险

系统失效时损失可见；系统恢复时如果没有流控与预热策略，容易出现“刚恢复就再次崩溃”的连锁反应，导致停机时间显著拉长。

⁴视频时间：00:48:12-00:49:00。

5.2 15 分钟恢复的关键动作

该案例中团队通过在服务前加流量调节 (traffic regulator)，限制早期放量速度，让副本逐步预热，最终在约 15 分钟内恢复全链路。这是典型的“以吞吐爬坡换稳定恢复”。

可迁移的 SRE 经验

- 冷启动阶段必须有 admission control；
- 队列长度与并发阈值要联动，不可静态配置；
- 恢复流程要脚本化，避免临场手工操作放大错误；
- 关键组件需具备 brownout 模式，先保核心能力再恢复完整能力。

Agent 系统的放大效应

Agent 的请求通常是多步链路，一个环节失控会放大到整条工作流。传统 API 服务可承受的短抖动，在 Agent 系统里可能演化为任务雪崩。

5.3 本章小结

生产系统的核心能力不是“平时跑得快”，而是“故障时不扩散、恢复时不复发”。Agent 场景下这一要求更高。

6 硬件与系统共设计：从 OCP 到 DGX

6.1 架构回摆：从分散微服务到高密度计算节点

讲者回顾了 Cray-2、OCP 服务器与现代 AI 机箱的演化。传统云强调模块化小节点，利于替换和弹性；AI 训练/推理要求高带宽互联与低通信延迟，又推动架构向高密度节点回摆。



图 5: AI 计算节点走向高密度化：DGX 类系统示意⁵

AI 负载下“模块化”与“集成化”的再平衡

模块化降低运维复杂度；集成化提升互联效率。AI 系统常需要在集成节点内部追求极致带宽，再在节点间做分层调度。

6.2 节点形态对软件栈的影响

当单机从“若干独立 CPU”变为“8 GPU + 2 CPU + 高速互联”，软件抽象也会变化：

- 资源单位不再只是“CPU 核 + 内存”，而是“拓扑感知的加速器组”；
- 调度器需要理解 NVLink / PCIe / NIC 布局；
- 容器编排需要加入通信亲和性约束。

为什么硬件细节重新进入应用层视野

过去很多应用团队不需要关心机架与交换机拓扑；在大模型任务里，物理拓扑直接影响训练时延、吞吐和成本。系统抽象不能完全屏蔽底层结构。

过度抽象的代价

如果平台把所有异构资源都抽成“统一 VM”，短期易用性更好，长期会掩盖通信瓶颈与拓扑冲突，导致成本优化空间被锁死。

6.3 本章小结

AI 基础设施正在经历“硬件-软件共设计”回归。高密度节点、拓扑感知调度和分层抽象将成为主流设计方向。

7 RAG 与 Agent 的关系：不是替代，而是内化

7.1 RAG “降温”的真实含义

在问答环节，讲者回应了“RAG 是否过时”：RAG 仍然关键，只是被更深地内嵌到产品栈中。用户不再显式讨论它，就像很少有人讨论数据库，但所有系统都在用数据库。

⁵视频时间：00:51:44-00:52:02。



图 6: 问答环节: RAG 由显式卖点转为隐式基础能力⁶

RAG 的地位变化: 从产品功能到系统部件

在 Agent 系统里, 检索能力常与工具调用、记忆机制、任务规划融合, 形成“检索-推理-执行”的联合闭环, 而非单一检索模块。

7.2 分阶段排序: 成本与准确率的联合优化

讲者使用推荐系统经验类比 RAG/检索流程: 先用便宜模型做粗排, 再用更强模型做精排。这个分层策略本质是“成本优先缩小候选集, 再对高价值候选投入高精度算力”。

两阶段检索-重排策略

- 第一阶段: 关键词、向量、规则过滤, 目标是召回率与成本可控;
- 第二阶段: LLM 重排与上下文推理, 目标是答案准确率与解释质量。

7.3 幻觉问题的边界: 常识与事实的分界线

讲者讨论了“模型记忆事实”与“上下文提供事实”的边界模糊性。常识和事实知识并非总能清晰切开, 这也是幻觉长期难解的重要原因。

企业场景必须把事实来源显式化

在企业系统中, 答案质量不能依赖隐性记忆。需要可追踪数据源、版本化检索索引、可复现实验与可解释引用链路, 才能把幻觉风险降到可管理范围。

⁶视频时间: 01:05:35-01:06:00。

7.4 本章小结

RAG 没有过时，而是成熟并内化。Agent 时代的关键不是“是否使用 RAG”，而是“如何把检索、推理、执行放入同一可治理管线”。

8 AI 云调度难题：拓扑、数据与抽象层冲突

8.1 局部性约束重新变成一等公民

在 1:09 左右，讲者指出 AI 训练任务对物理邻近性高度敏感。请求四台机器时，最好位于同机架并挂在同一交换机上，否则通信代价会吞噬算力收益。



图 7: 分布式训练中的机架/交换机局部性约束⁷

旧云抽象与新负载矛盾

传统云强调资源可替换与位置无关；AI 训练强调位置相关和链路稳定。两者目标不一致，导致通用抽象在 AI 场景下出现性能与成本折损。

8.2 数据面复杂度：不只是“装个框架”

讲者提到，安装 PyTorch 很快，但把 PB 级数据接入目标存储与训练作业才是长期瓶颈。不同环境（对象存储、块存储、并行文件系统）在吞吐、延迟与一致性上差异巨大，数据管线设计成为系统成败关键。

⁷ 视频时间：01:09:40–01:09:53。

数据面工程的三个核心动作

1. 数据布局：冷热分层、分片策略、预取窗口；
2. 数据通路：对象存储到训练节点的高效拉取与缓存；
3. 数据治理：版本、血缘、质量校验与回放能力。

层面	传统云默认假设	AI 训练现实	改造方向
调度	位置无关，可替换 VM	位置相关，拓扑敏感	拓扑感知调度器
存储	统一接口即可	多存储栈性能差异显著	分层数据管线
网络	服务间 RPC 为主	AllReduce / 参数同步密集	高带宽低抖动网络
运维	微服务弹性扩缩容	训练作业长时、失败代价高	作业级容错与恢复

表 2: AI 负载下“抽象便利性”与“物理现实”的矛盾

平台团队常见低估：忽略数据搬运成本

很多团队在成本估算时只看 GPU 单价，忽略数据准备、跨区传输、存储读放大和作业重试成本，导致总成本明显高于预期。

8.3 本章小结

AI 系统设计正在从“计算资源管理”扩展为“计算 + 数据 + 拓扑”协同优化问题。调度与数据面能力将决定平台竞争力。

9 创业与产业判断：价值优先、垂直突破、效率回归

9.1 创业切入点：基础模型之外的价值密度

在 Q&A 中，讲者强调从零训练新基础模型仍有空间，但门槛极高，需要资本、人才与算力协同。更现实且机会更大的方向，是结合行业知识在企业场景做垂直化产品，把通用模型能力转化为可交付价值。

创业建议（工程视角）

把资源集中在“高价值垂直问题”，而不是盲目复制通用模型路线。差异化来自行业数据、流程重构与系统集成深度。

9.2 中心化与边缘化的动态平衡

讲者提出一个值得长期追踪的张力：一方面，大规模中心化集群持续推动上限；另一方面，机器人、手机端智能、CDN/电信边缘节点推动高效模型和分布式推理需求。



图 8: 边缘 AI 与分布式推理的产业方向⁸

探索 (Exploration) 与利用 (Exploitation)

- 探索：追求更强模型上限，可容忍更高成本；
- 利用：追求可部署效率，强调延迟、功耗与单位成本；
- 工业系统通常需要两条路线并行，不存在单一最优策略。

9.3 价值创造与风险边界

讲者对产业持审慎乐观态度：只要系统持续创造用户价值，成本曲线可以被工程优化逐步消化。但若缺乏真实价值，系统最终会成为“成本驱动的纸牌屋”。

价值缺失是最大结构性风险

如果产品无法在搜索、编码、生产力或企业流程中形成稳定增益，任何技术叙事都难以持续。系统设计的终点必须回到业务价值闭环。

9.4 本章小结

未来几年会同时出现“超大中心集群”与“高效边缘推理”两条路线。创业公司更适合在垂直场景中把通用能力转化为可量化价值。

⁸视频时间：01:16:26–01:16:41。

10 工程落地清单：面向 Agent 系统的设计原则

10.1 从课程内容抽象出的十条实践原则

为了把讲者的系统观点转化为团队可执行方法，本节整理出面向 Agent 平台建设的十条工程原则：

1. 把任务链路拆成可观测模块，避免黑箱联动；
2. 先定义恢复策略，再上线高并发流量；
3. 在系统设计早期就引入拓扑感知调度；
4. 数据面设计与算力规划同步推进；
5. 检索与重排要分层，先控成本再控准确；
6. 企业场景必须做事实来源追踪；
7. 把 SLA 与成本指标纳入同一优化目标；
8. 把“模型升级”与“系统升级”解耦；
9. 为边缘部署预留轻量模型路径；
10. 持续做演练：故障恢复、流量峰值、链路退化。

行动建议

若团队资源有限，应优先补“可观测性 + 恢复能力 + 数据管线”三件事。这三者是把实验系统推向生产系统的最短路径。

10.2 评估框架

维度	评估问题	观测指标	常见改进手段
可靠性	故障后多久恢复？是否二次崩溃？	MTTR、错误峰值、重试风暴次数	流控、预热、分级降级
成本效率	单任务成本是否可预测？	Token 成本、GPU 利用率、单位任务毛利	分阶段推理、缓存、模型路由
数据质量	检索事实是否可追溯？	引用覆盖率、事实冲突率	数据版本化、检索评测
可扩展性	高并发下是否退化平滑？	P95/P99 延迟、队列积压长度	并发控制、弹性策略
部署灵活性	能否同时支持云端与边缘？	端侧延迟、功耗、模型体积	蒸馏、量化、异构调度

为什么评估框架必须“系统化”

Agent 质量不是单指标问题。只看准确率会忽略稳定性与成本，只看成本会牺牲可靠性。系统化评估能减少局部最优导致的整体失效。

10.3 本章小结

课程中的方法论可以直接转化为工程清单：先建立系统能力，再追求模型能力红利，才能实现可持续迭代。

11 案例推演：企业级 Agent 平台的分阶段建设路线

11.1 阶段 0：先做“可控的最小系统”

很多团队上来就尝试多 Agent、复杂工具编排和自动化闭环，结果在第一周就被日志噪声、链路超时和错误重试拖垮。更务实的路径是先构建一个可控最小系统（Minimum Controllable System）：

- 单 Agent + 单业务域 + 可回放日志；
- 检索源有限且可标注；
- 失败默认安全（fail-safe）而不是强行继续执行。

最小系统的目标不是“能力上限”，而是“行为确定性”

只有当系统在固定输入下能稳定复现，后续优化才有意义。否则每次改动都无法判断收益来自模型提升还是偶然波动。

11.2 阶段 1：把检索与执行解耦

课程里强调 RAG 与 Agent 并非替代关系。在工程实现上，建议把“知识检索”与“动作执行”解耦为两个独立层：

1. 检索层负责召回候选事实与证据；
2. 决策层负责选择工具与下一步动作；
3. 执行层负责副作用操作并返回结构化结果。

解耦后的直接收益是：可以分别评估召回质量、规划质量和执行成功率，避免把所有问题都归因给“模型幻觉”。这也更符合讲者提出的系统化思路。

推荐的最小观测指标

- 检索召回率（Top-k Evidence Recall）；
- 规划一致性（同问题多次执行的动作序列偏差）；
- 工具成功率（成功调用比例与平均重试次数）；
- 端到端任务成功率（按业务验收标准定义）。

11.3 阶段 2：引入流控与恢复策略

在 48 分钟案例里，系统恢复失败本质是“恢复流量 > 服务恢复速度”。因此平台第二阶段的关键不是再堆功能，而是把流控和恢复设计前置。一个可执行的方案是：

- 对每类任务设定并发预算（concurrency budget）；
- 对模型服务设定暖启动窗口（warm-up window）；
- 对下游工具调用设定熔断阈值（circuit breaker）；
- 对任务状态设定幂等恢复点（idempotent checkpoint）。

没有恢复策略的自动化等于放大器

自动化链路会把单次错误放大到批量错误。对于企业场景，“先恢复、再优化”比“先加功能、再救火”的总成本更低。

11.4 阶段 3：治理与合规上线

当系统进入企业生产后，关注点会从“能不能做”转向“能不能长期可控地做”。这要求引入治理层：

- 权限与审计：谁触发了什么动作，证据链是否完整；
- 数据边界：私有数据是否跨域泄漏；
- 合规约束：是否满足行业监管与留痕要求；
- 运营指标：成本、成功率、时延是否可持续。

建设阶段	核心目标	必须交付物	退出条件
阶段 0	行为可复现	结构化日志、回放脚本、基线任务集	同输入结果稳定且可解释
阶段 1	模块可诊断	检索/规划/执行分层指标面板	能定位性能与质量瓶颈来源
阶段 2	故障可恢复	流控、熔断、幂等恢复点	峰值下无级联崩溃
阶段 3	生产可治理	权限审计、数据边界、合规报告	满足业务与监管上线标准

表 4: 企业级 Agent 平台从 PoC 到生产的四阶段路线图

11.5 本章小结

平台建设应该遵循“可控性先于复杂度”的路径。先把复现、诊断、恢复、治理补齐，再追求更高自动化水平，才能把 Agent 变成长期可运营能力。

12 术语与指标附录：把系统讨论落到可执行口径

12.1 关键术语对照

为避免跨团队沟通歧义，这里把课程中涉及的核心概念统一为工程口径：

术语	课程语境	工程落地定义
Agentic AI	模型具备多步执行与工具调用能力	具状态、可规划、可执行、可恢复的任务系统
RAG	外部知识增强生成	可追溯检索 + 证据注入 + 答案生成管线
Data Locality	数据与算力的空间邻近性	任务调度时的数据/网络拓扑约束
Exploration	冲刺模型能力上限	容忍高成本的试验型研发阶段
Exploitation	追求部署效率与收益	以单位成本和稳定性为主导的生产阶段

表 5: 课程高频术语的工程化定义

12.2 建议的周报指标模板

如果团队已经在做 Agent 平台迭代，推荐每周至少跟踪以下指标，并按“质量、效率、稳定性”三类汇报：

指标类别	指标	建议频率	目标趋势
质量	任务成功率、事实一致率、误操作率	日/周	成功率上升，误操作下降
效率	平均成本/任务、GPU 利用率、缓存命中率	日/周	成本下降，利用率上升
稳定性	P95 延迟、错误峰值、MTTR	日/周	尾延迟与 MTTR 下降
治理	可审计覆盖率、权限违规次数	周/月	审计覆盖率上升，违规归零

为什么要把指标写进笔记

课程知识只有转化为团队的例行观测和决策机制，才会变成真实能力。指标模板的作用是让“系统设计”进入持续迭代闭环，而不是停留在一次性讨论。

12.3 本章小结

统一术语和指标口径，是跨研究、平台、产品团队协作的最低成本手段。它直接决定系统优化是否可持续、可复盘、可扩展。

13 总结与延伸

13.1 全讲核心结论

主题	关键结论	对实践的直接影响
系统演进主线	AI 工作负载迫使基础设施重写	需要拓扑感知调度与数据面重构
浪潮切换	RAG 等能力从显式热点走向隐式基础设施	重点从概念竞争转向系统集成能力
生产可靠性	故障恢复阶段最容易触发级联崩溃	必须建设流控、预热、降级、演练机制
硬件软件关系	高密度节点与高带宽互联成为主流	平台抽象需暴露关键物理约束
创业机会	垂直企业场景仍有大量未解问题	结合行业知识做高价值差异化交付
未来方向	中心化大模型与边缘高效推理并存	双路线并行: 上限探索 + 效率利用

表 7: Lecture 10 的系统设计结论与工程含义

一句话总结

Agent 时代的竞争，不再只是“谁的模型更强”，而是“谁能把模型能力稳定、低成本、可治理地变成真实系统能力”。

13.2 进一步阅读

- Berkeley RDI Agentic AI MOOC F25 其余讲次（安全、评测、多智能体、生产部署）。
- Snowflake 与 Databricks 的 Data Cloud 架构公开资料。
- NVIDIA DGX 平台与大规模训练系统工程文档。
- 云原生与 AI 基础设施融合实践（Kubernetes, Slurm, Ray, SkyPilot 等）。
- 企业级 RAG/Agent 评测方法（事实性、可追溯性、成本与时延联合评估）。